Informatique Théorique

CM 11: Programmation dynamique (2)

Florian Bridoux

Polytech Nice Sophia

2025-2026

Table des matières

1. Problème d'optimisation
2. Coupe optimale d'une barre (*Rod Cutting*)
3. Solution optimale
4. Compter le nombre de solutions optimales
5. Optimisations

Programmation dynamique et problèmes d'optimisation

Les problèmes d'optimisation :

- Ce sont des problèmes qui admettent un ensemble de solutions S.
- On a $c : S \to \mathbb{R}$ une fonction objectif.
- On cherche une solution s^* telle que $c(s^*) = max\{c(s) : s \in S\}$ si c'est un problème de maximisation.
- On change *max* par *min* si c'est un problème de minimisation.

Table des matières

1. Problème d'optimisation
2. Coupe optimale d'une barre (*Rod Cutting*)
3. Solution optimale
4. Compter le nombre de solutions optimales
5. Optimisations

Coupe optimale d'une barre (*Rod Cutting*)

Le problème de coupe optimale d'une barre :

On dispose d'une barre de longueur n et de prix pour chaque longueur entière de morceau. L'objectif est de découper la barre en morceaux de manière à maximiser les revenus.

Coupe optimale d'une barre (*Rod Cutting*)

- **Nom** : Coupe optimale d'une barre (Rod Cutting).
- **Entrée** :
  - Un entier $n \geq 0$ encodé en binaire représentant la longueur de la barre ;
  - un tableau d'entiers p de taille n, où $p[i-1] > 0$ représente le prix de vente d'un morceau de longueur i.
- **Solutions admissibles** : une liste `lst` d'entiers naturels représentant la taille des morceaux :

$$\sum_{m_i \in \texttt{lst}} m_i = n.$$

- **Fonction objectif** (à maximiser) : le revenu total obtenu par la vente des morceaux. Autrement dit :

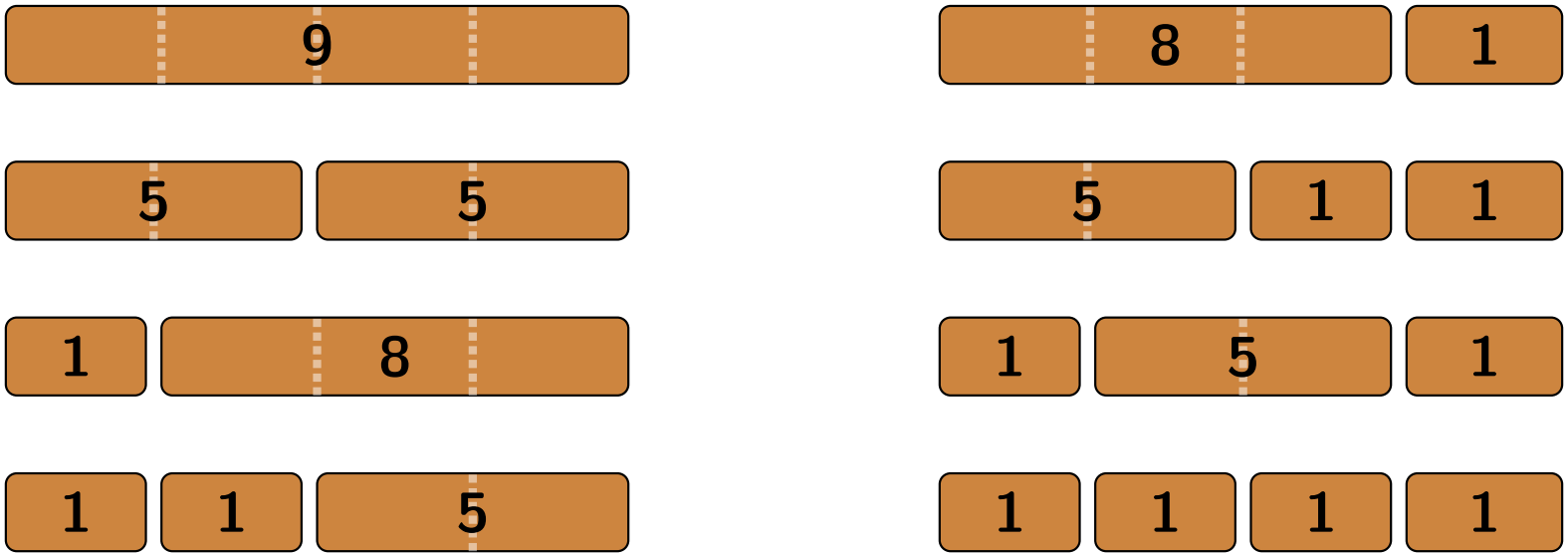
$$c(t) = \sum_{m_i \in \texttt{lst}} p[m_i - 1].$$

Coupe optimale d'une barre (*Rod Cutting*)

Prenons $n = 4$ et les prix des morceaux :



On considère tous les découpages possibles d'une planche de taille 4 :



Meilleur découpage : deux morceaux de taille 2, ce qui donne un prix total de 5 !

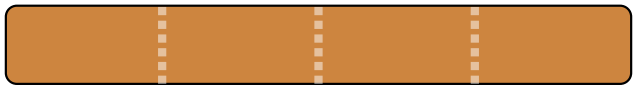
Algorithme où on liste les découpages

```
def decoupages(n):
    if n==0:
        return [[]]
    lst_decoupages = []
    for i in range(1,n+1):
        for lst in decoupages(n-i):
            lst_decoupages.append([i]+lst)
    return lst_decoupages
def prix(p,lst):
    return sum( p[m_i-1] for m_i in lst)
def meilleur_prix(n,p):
    return max(
    prix(p,lst) for lst in decoupages(n))
print(decoupages(4)) #[[1, 1, 1, 1],
#[1, 1, 2], [1, 2, 1], [1, 3], [2, 1, 1], [2, 2],
#[3, 1], [4]]
print(meilleur_prix(4,[1,5,8,9])) #10
```

Algorithme où on liste les découpages

C'est un très mauvais algorithme :

- On charge en mémoire tous les découpages en même temps et donc même les techniques de mémoïsation ne serviraient à rien : la complexité en mémoire additionnel et en temps est au moins le nombre de découpages.
- Il y a 2^{n-1} découpages différents. En effet sur chaque ligne pointillé de la barre de taille n on a deux choix : faire une découpe ici ou non. Et sur une barre de taille n il y a $n-1$ lignes en pointillé.



Variante avec une énumération

```
def decoupages(n):
    if n==0:
        yield []
    for i in range(1,n+1):
        for lst in decoupages(n-i):
            yield [i]+lst

def prix(p,lst):
    return sum( p[m_i-1] for m_i in lst)

def meilleur_prix(n,p):
    return max(
    prix(p,lst) for lst in decoupages(n))
```

Variante itérative

```
from itertools import product
def decoupages(n):
    for masque in product([1, 0], repeat=n-1):
        lst = []
        i=0
        for j in range(1,n+1):
            if j==n or masque[j-1]==1:
                lst.append(j-i)
                i=j
        yield lst

def prix(p,lst):
    return sum( p[m_i-1] for m_i in lst)

def meilleur_prix(n,p):
    return max(
    prix(p,lst) for lst in decoupages(n))
```

Variante avec une énumération

- Ces deux variations permettent de ne pas charger toute la liste des découpages en mémoire mais elles ne résolvent pas le problème principal qui est l'itération sur toutes les découpages.
- C'est là qu'on peut utiliser le fait que c'est un problème d'optimisation : on ne veut pas tous les prix, on veut seulement connaître le meilleur prix.

Version avec memoïsation

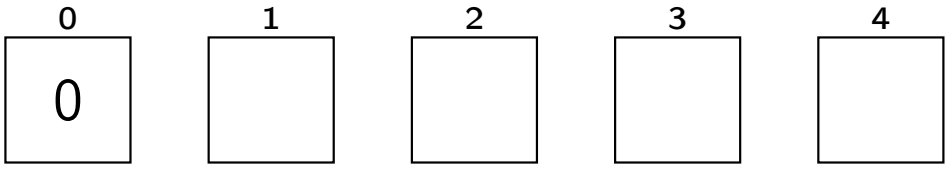
Version naïve :

```
def meilleur_prix(n,p):
    if n==0:
        return 0
    return max( p[i-1]+meilleur_prix(n-i,p)
    for i in range(1,n+1) )
```

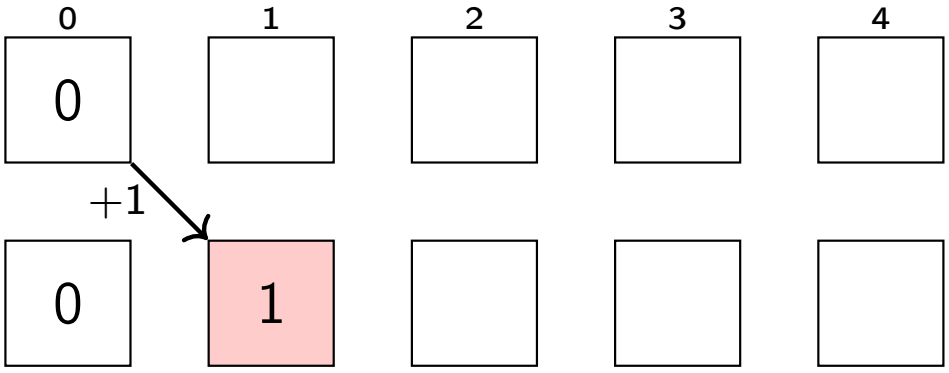
Avec mémoïsation :

```
def meilleur_prix(n,p,dct=None):
    if dct == None:
        dct = {0:0}
    if n not in dct:
        dct[n] = max( p[i-1]+
        meilleur_prix(n-i,p,dct)
        for i in range(1,n+1) )
    return dct[n]
```

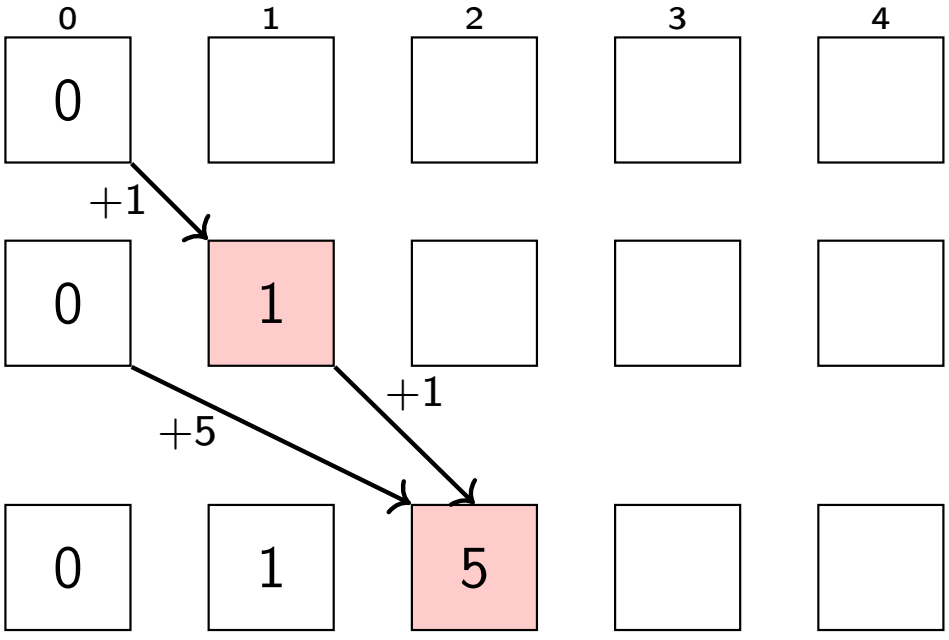
Version avec tabulation



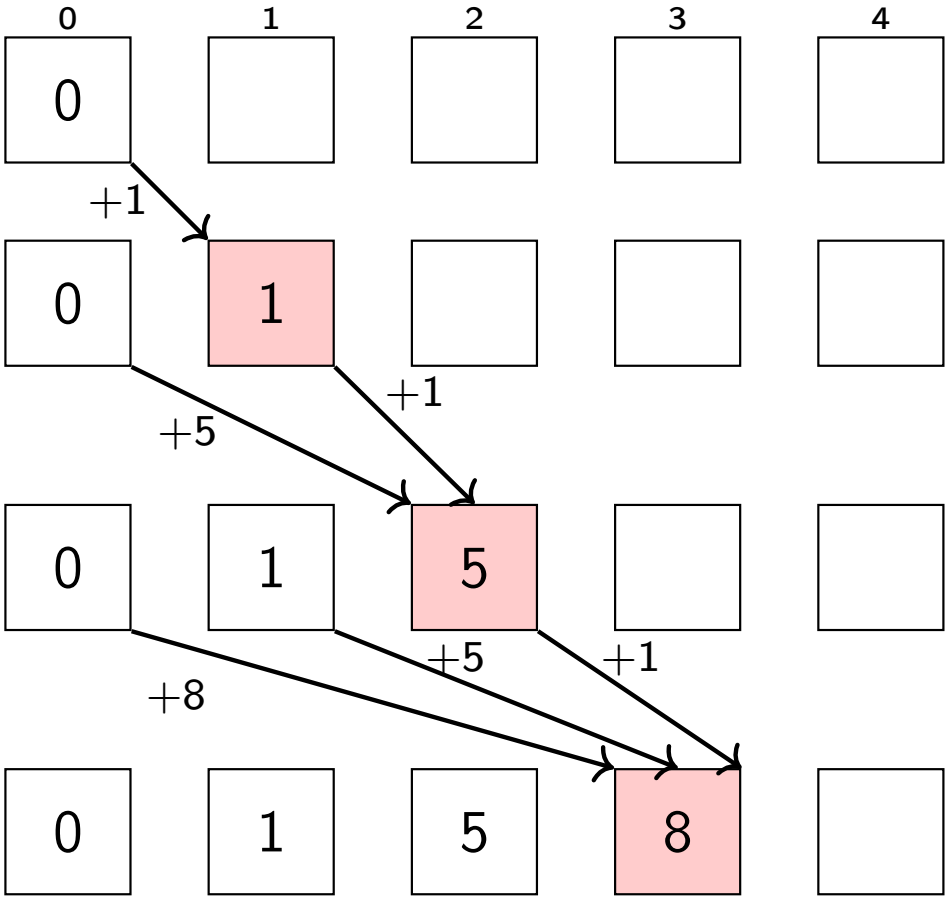
Version avec tabulation



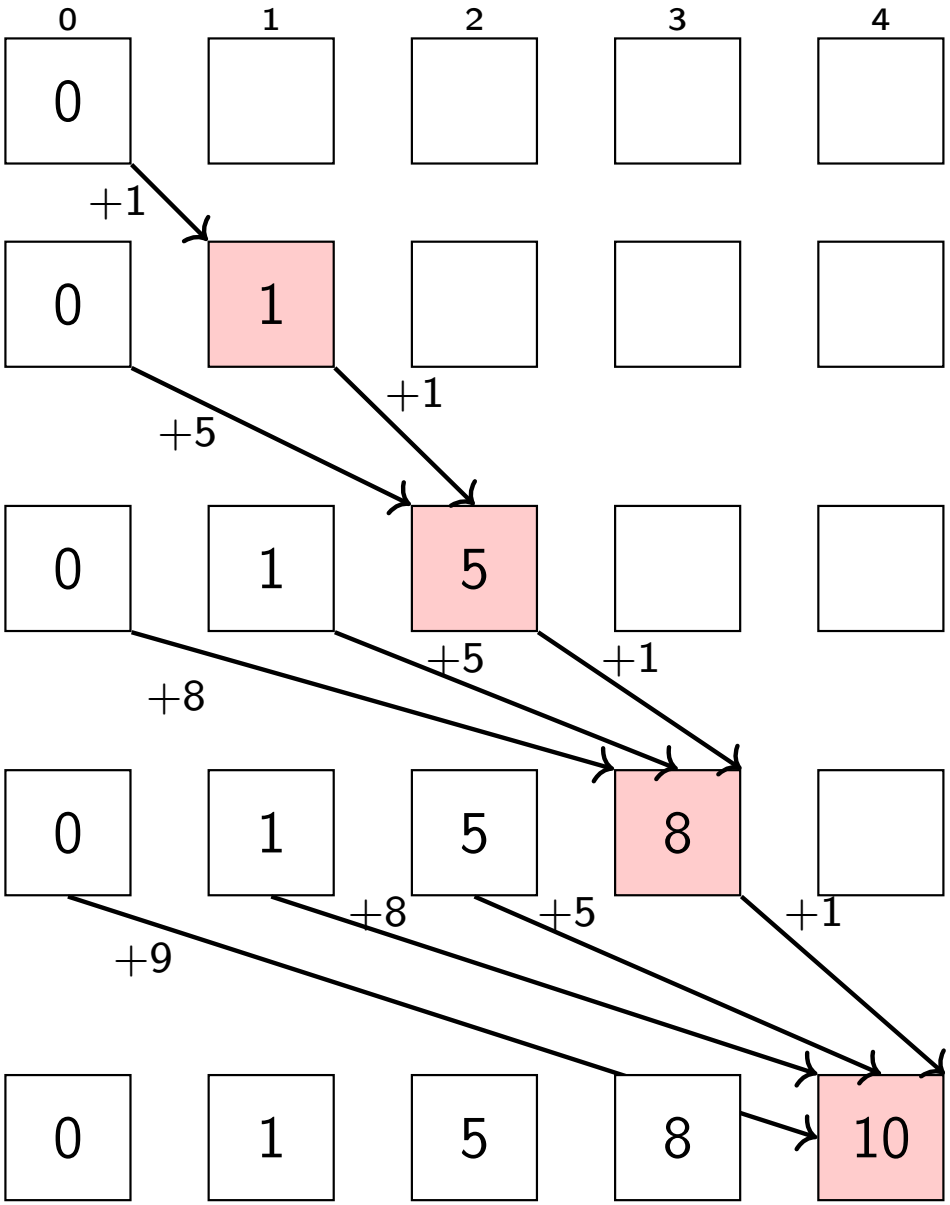
Version avec tabulation



Version avec tabulation



Version avec tabulation



Version avec tabulation

Version naïve :

```
def meilleur_prix(n,p):
    if n==0:
        return 0
    return max( p[i-1]+meilleur_prix(n-i,p)
    for i in range(1,n+1) )
```

Avec tabulation :

```
def meilleur_prix(n,p):
    tab = [0]*(n+1)
    for j in range(1,n+1):
        tab[j] = max( p[i-1]+tab[j-i]
        for i in range(1,j+1))
    return tab[n]
```

Version avec tabulation

- On voit que si on considère que les additions se font en $O(1)$, la complexité (avec mémoïsation et avec tabulation) est en $O(n^2)$ pour le temps et $O(n)$ en espace.

Table des matières

1. Problème d'optimisation
2. Coupe optimale d'une barre (*Rod Cutting*)
3. Solution optimale
4. Compter le nombre de solutions optimales
5. Optimisations

Solution optimale

Comment récupérer une solution optimale (et pas juste sa valeur) ?

Solution 1 : On calcule une table des solutions optimales en même temps que la table des valeurs optimales :

```
def meilleure_solution(n,p):
    tab = [0]*(n+1)
    solutions = [[] for _ in range (n+1)]
    for j in range(1,n+1):
        for i in range(1,j+1):
            if tab[j] < p[i-1]+tab[j-i]:
                tab[j] = p[i-1]+tab[j-i]
                solutions[j] = solutions[j-i]+[i]
    return solutions[n]
```

Problème : surcoût en temps et en mémoire. Ici, on passe de $O(n^2)$ à $O(n^3)$ en temps et $O(n)$ à $O(n^2)$ en mémoire.

Solution optimale

Solution 2 : On calcule d'abord la table et ensuite on recréé une solution optimale dans une approche *top-down*.

```
def meilleure_solution(n,p):
    tab = [0]*(n+1)
    for j in range(1,n+1):
        tab[j] = max( p[i-1]+tab[j-i]
        for i in range(1,j+1))

    j,solution = n,[]
    while j > 0:
        i_max = max( range(1, j + 1),
        key=lambda i: p[i-1] + tab[j-i] )
        solution.append(i_max)
        j-=i_max
    return list(reversed(solution))
```

La complexité $O(n^2 + n) = O(n^2)$ en temps et $O(n + n) = O(n)$ en mémoire.

Table des matières

1. Problème d'optimisation
2. Coupe optimale d'une barre (*Rod Cutting*)
3. Solution optimale
4. Compter le nombre de solutions optimales
5. Optimisations

Solution optimale

Solution 2 : On calcule d'abord la table et ensuite on recréé une solution optimale dans une approche *top-down*.

```
def nb_solutions_optimales(n,p):
    tab = [0]*(n+1)
    nb  = [0]*(n+1)
    nb[0] = 1

    for j in range(1,n+1):
        for i in range(1,j+1):
            val = p[i-1] + tab[j-i]
            if val > tab[j]:
                tab[j] = val
                nb[j] = nb[j-i]
            elif val == tab[j]:
                nb[j] += nb[j-i]

    return nb[n]
```

La complexité $O(n^2 + n) = O(n^2)$ en temps et $O(n + n) = O(n)$

Table des matières

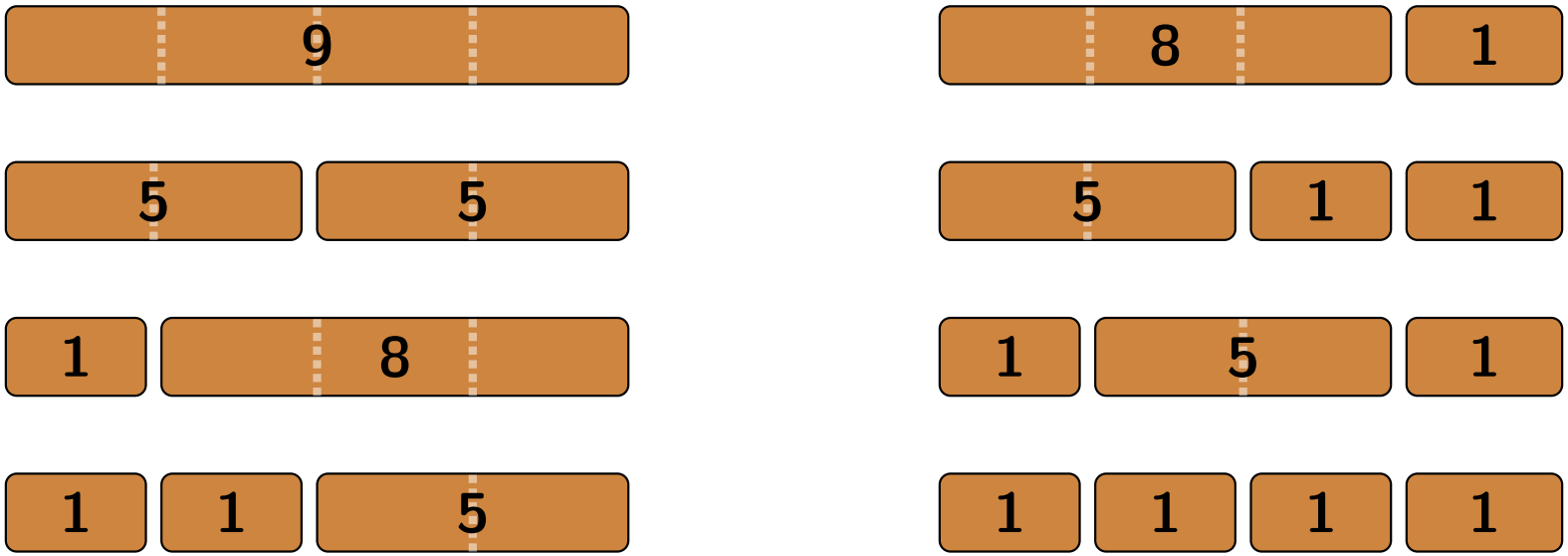
1. Problème d'optimisation
2. Coupe optimale d'une barre (*Rod Cutting*)
3. Solution optimale
4. Compter le nombre de solutions optimales
5. Optimisations

Coupe optimale d'une barre (*Rod Cutting*)

Prenons $n = 4$ et les prix des morceaux :



On considère tous les découpages possibles d'une planche de taille 4 :



On se rend compte qu'il y a plusieurs découpages équivalents : `[5,1,1]`, `[1,5,1]`, `[1,1,5]` ou `[8,1]`, `[1,8]`.

Coupe optimale d'une barre (*Rod Cutting*)

- **Nom** : Coupe optimale d'une barre (Rod Cutting).
- **Entrée** :
  - Un entier $n \geq 0$ encodé en binaire représentant la longueur de la barre ;
  - un tableau d'entiers p de taille n, où $p[i-1] > 0$ représente le prix de vente d'un morceau de longueur i.
- **Solutions admissibles** : un tableau t d'entiers naturels de taille n tel que $t[i]$ est le nombre de morceaux de longueur i obtenus après découpe, et tel que

$$\sum_{1 \leq i \leq n} i \cdot t[i] = n.$$

- **Fonction objectif** (à maximiser) : le revenu total obtenu par la vente des morceaux. Autrement dit :

$$c(t) = \sum_{1 \leq i \leq n} t[i] \cdot p[i-1].$$

Optimisation : approche naïve

Approche naïve :

```
def meilleur_prix_naif(n, p, k=None):
    if k == None:
        k = n
    if n == 0:
        return 0
    if k == 0:
        return float('-inf')
        # impossible de decouper n > 0
    # option 1 : ne pas utiliser k
    res = meilleur_prix_naif(n, p,k-1)
    # option 2 : utiliser k
    if k <= n:
        res = max(res, p[k-1]
        + meilleur_prix_naif(n-k, p,k))
    return res
```

Optimisation : approche naïve

Complexité : On passe d'une complexité de d'environ 2^{n-1} à $2^{\Theta(\sqrt{n})}$ environs (voir le théorème de Hardy–Ramanujan sur le nombre de partitions d'un entier pour plus de précisions).

Optimisation : mémoïsation

```
def meilleur_prix_memo(n, p,k=None, memo=None):
    if memo is None:
        k=n
        memo = {}
    if (n, k) in memo:
        return memo[(n, k)]
    if n == 0:
        return 0
    if k == 0:
        return float('-inf')
    res = meilleur_prix_memo(n, p,k-1, memo)
    if k <= n:
        res = max(res, p[k-1] +
        meilleur_prix_memo(n-k,p, k,  memo))

    memo[(n, k)] = res
    return res
```

Optimisation : tabulation

```
def meilleur_prix_tab(n, p):
    tab = [[0]*(n+1) for _ in range(n+1)]

    for k in range(1, n+1):
        for j in range(1, n+1):
            # ne pas prendre k
            tab[j][k] = tab[j][k-1]

            # prendre k si possible
            if k <= j:
                tab[j][k] = max(tab[j][k],
                p[k-1] + tab[j-k][k])

    return tab[n][n]
```

Optimisation : tabulation

On ne gagne absolument rien avec cette approche. On passe de $O(n^2)$ à $O(n \times |p|)$ c'est-à-dire à $O(n^2)$. Cela vient du fait que certaines cases calculés sont inutiles.